(c) Marçal, J.

VETORES, CONTADORES, ACUMULADORES, TIPOS DADOS ABSTRATOS (TDA), TIPOS DE DADOS ESTRUTURADOS (TDE)

TANENBAUM, Aaron. **ESTRUTURA DE DADOS USANDO C**. BRASIL: MAKRON, 1995.

(c) Marçal, J.

VETORES/ARRAYS

(c) Marçal, J.

VETORES / ARRAYS

- **ENTENDENDO VETORES**
  - **ENTENDENDO –** Os vetores ou também conhecidos como arrays, são estruturas lineares, com dimensões pré-definidas e de um determinado tipo de dado.
  - **ESTRUTURA** – Um determinado vetor deve conter uma estrutura básica composta por:
    - Nome;
    - Tipo de dado;
    - Tamanho (podendo iniciar em 0 ou 1, dependendo da linguagem);
    - Índice de cada posição;

(c) Marçal, J.

VETORES / ARRAYS

- **ENTENDENDO VETORES**
  - **FUNÇÕES BÁSICAS –** Os vetores são estruturas estáticas que permite o armazenamento de diferentes tipos de dados, são altamente poderosos e podem ser utilizados em inúmeros tipos de aplicações. Para que estas estruturas possam ser utilizadas, algumas operações se fazem necessárias:
    - **CRIAÇÃO/DEFINIÇÃO** – Momento onde a estrutura do vetor é definida e alocado o espaço na memória.
    - **INSERÇÃO** – Responsável por adicionar elementos/valores;
    - **PESQUISA** – Processo que permite localizar um determinado valor no vetor.
    - **REMOÇÃO** – Operação responsável pela remoção de um valor qualquer do vetor.

(c) Marçal, J.

VETORES / ARRAYS

- **EXEMPLO**
  - **IDADES –** Iremos agora criar um vetor responsável por armazenar diferentes Registros Acadêmicos (RA's) fornecidos pelo usuário.
  - **ETAPAS –** Seguem abaixo as etapas para desenvolvimento do exemplo:
    - DEFINIÇÃO – a) Definir a variável com 50 posições responsável pelo vetor; b) Criar a variável responsável por controlar as opções do menu; c) Criar variável responsável por controlar o índice do vetor a ser utilizado.
    - MENU – Criar o menu com as opções Inserir, Pesquisar e Remover.
    - CÓDIGO – Desenvolver os códigos referente a cada opção do menu.

(c) Marçal, J.

CONTADORES e ACUMULARES

(c) Marçal, J.

CONTADORES e ACUMULADORES

- **ACUMULADOR**
  - **ENTENDENDO –** Um acumulador nada mais é que uma variável responsável por armazenar valores (acumular), normalmente este conceito é utilizado quando se deseja acumular valores a partir de uma ocorrência ou regra;
  - **EXEMPLOS** – Baseado em um vetor (vetIdades[4]) de inteiros.

```c
int main(int argc, char *argv[]) {
    /* declaração */
    int vetorIdades[4];

    /* atribuição de valores */
    vetorIdades[0] = 15; vetorIdades[1] = 25;
    vetorIdades[2] = 48; vetorIdades[3] = 32;

    /* acumulador: Somar as idades */
    int acum = 0; int i = 0;
    for (i = 0; i < 4; i++) {
        acum = acum + vetorIdades[i];
    }

    /* impressao */
    printf("A soma e: %i", acum);

    return 0;
}
```

```c
int main(int argc, char *argv[]) {
    /* declaração */
    int vetorIdades[4];

    /* atribuição de valores */
    vetorIdades[0] = 15; vetorIdades[1] = 25;
    vetorIdades[2] = 48; vetorIdades[3] = 32;

    /* acumulador: Somar as idades maiores 18 */
    int acum = 0; int i = 0;
    for (i = 0; i < 4; i++) {
        if (vetorIdades[i] > 18) {
            acum = acum + vetorIdades[i];
        }
    }

    /* impressao */
    printf("A soma das idades > 18 e: %i", acum);

    return 0;
}
```

(c) Marçal, J.

CONTADORES e ACUMULADORES

- **CONTADOR**
  - **ENTENDENDO –** Um contador trata-se de uma variável responsável por contar o número de ocorrências de um determinado evento desejado;
  - **EXEMPLOS** – Baseado em um vetor (vetIdades[4]) de inteiros.

```c
int main(int argc, char *argv[]) {
    /* declaração */
    int vetorIdades[4];

    /* atribuição de valores */
    vetorIdades[0] = 15; vetorIdades[1] = 25;
    vetorIdades[2] = 48; vetorIdades[3] = 32;

    /* contador: Contar as idades */
    int cont = 0; int i = 0;
    for (i = 0; i < 4; i++) {
        cont = cont + 1;
    }

    /* impressao */
    printf("O num. idades e: %i", cont);

    return 0;
}
```

```c
int main(int argc, char *argv[]) {
    /* declaração */
    int vetorIdades[4];

    /* atribuição de valores */
    vetorIdades[0] = 15; vetorIdades[1] = 25;
    vetorIdades[2] = 48; vetorIdades[3] = 32;

    /* contador: Contar as idades maiores 18 */
    int cont = 0; int i = 0;
    for (i = 0; i < 4; i++) {
        if (vetorIdades[i] > 18) {
            cont = cont + 1;
        }
    }

    /* impressao */
    printf("O num. idades > 18 e: %i", cont);

    return 0;
}
```

(c) Marçal, J.

MÉTODOS DA CLASSE e/ou TIPOS DE DADOS ABSTRATOS (TDA)

(c) Marçal, J.

MÉTODOS DA CLASSE e/ou TDA

- **ENTENDENDO**
  - **ENTENDENDO –** Os Tipos Abstratos de Dados (TDAs), trata-se de uma estrutura capaz de controlar através de métodos e operações, uma determinada estrutura.
  - **ABSTRAÇÃO** – Uma TDA deve conter os métodos responsáveis pela manipulação da estrutura em questão, ou seja, os usuários apenas irão acessa os métodos disponíveis, sem saber como a operação interna é realizada.

(c) Marçal, J.

MÉTODOS DA CLASSE e/ou TDA

- **ENTENDENDO**
  - **ENCAPSULAMENTO –** Os Tipos Abstratos de Dados (TDAs), encapsulam o código responsável pela manipulação, deixando visível para os usuários apenas as operações disponíveis.
  - **AUTO SUFICIENCIA** – Os TDAs, devem ser auto-suficientes, ou seja, conter métodos e propriedades suficientes para permitir todas as operações possíveis dentro da estrutura disponibilizada.

(c) Marçal, J.

MÉTODOS DA CLASSE e/ou TDA

- **EXEMPLOS**

```c
int somar(int a, int b) {
    return a + b; }

int subtrair(int a, int b) {
    return a - b; }

int main(int argc, char *argv[]) {
    int num1 = 25;  int num2 = 32;
    int resul = 0;

    resul = somar(num1, num2);
    printf("A soma e: %i", resul);

    resul = subtrair(num1, num2);
    printf("A subtracao e: %i", resul);

    return 0;
}
```

```c
int somarVetor(int vet[4]) {
    int soma = 0, i = 0;
    for (i = 0; i < 4; i++) {
        soma = soma + vet[i];
    }
    return soma;
}

int main(int argc, char *argv[]) {
    int x = 0; int idades[4];

    idades[0] = 10; idades[1] = 11;
    idades[2] = 21; idades[3] = 22;

    x = somarVetor(idades);
    printf("A soma e: %i", x);

    return 0;
}
```

(c) Marçal, J.

TIPOS DE DADOS ESTRUTURADOS (TDE)

(c) Marçal, J.

- **ENTENDENDO TDE**
  - **EXTENDENDO OS TIPOS PRIMITIVOS –** Os tipos de dados TDE, são uma extensão dos tipos de dados primitivos que podem representar qualquer coisa do mundo real no mundo virtual, tornando assim a programação mais realista quando comparada ao mundo real.
  - **CHAVES** – É comum a existência de um identificador único em cada tipo de dado estruturado, este identificador único permite a distinção entre os tipos, pois todos contem a mesma estrutura.

(c) Marçal, J.

TDE

- **ENTENDENDO TDE**
  - **CRIANDO –** Os tipos de dados estruturados devem ser definidos (criada sua estrutura) antes de serem utilizados, uma vez criada a estrutura, eles passam a poder serem utilizados como os demais tipos existentes.
  - **DEFINIÇÃO DA CHAVE** – Em geral é comum que o tipo de dado criado contenha um identificador que seja único, caso seja criado um TDE para representar um Aluno, o RA poderia facilmente ser o identificador único.

ENTENDENDO TDE

- **EXEMPLO –** "A empresa XPTO solicitou o desenvolvimento de um sistema para realização de todos os dados cadastrais de altura, peso e idade dos funcionários da empresa.". Neste caso, o foco principal são as informações pessoais dos funcionários.

```
4  struct infopes {
5      float altura;
6      float peso;
7      int idade;
8  };
9
10 int main(int argc, char *argv[]) {
11     struct infopes dados;
12
13     dados.altura = 1.78;
14     dados.peso = 84.7;
15     dados.idade = 37;
16
17     printf("Os dados sao peso: %.2f, altura: %.2f e idade %i.",
18       dados.altura, dados.peso, dados.idade);
19
20     return 0;
21 }
```

(c) Marçal, J.

TDE

- **TDE x CLASSES (Java)**
  - **DECLARAÇÃO e USO –** Em Java, para que possamos criar um TDE é necessário criarmos uma Classe, esta classe então passa a representar algo do mundo real.
  - **INSTANCIANDO** – Por ser uma classe, ao definir a uma variável o tipo TDE criado, esta variável precisa ser inicializada (através do comando new).

(c) Marçal, J.

PRÁTICA

- **EXEMPLO PRÁTICO 01 - CALCULADORA**
  - **CENÁRIO –** Desenvolva uma aplicação (menu) capaz de realizar as quatro operações básicas de uma calculadora, desenvolva uma TDA responsável pelos cálculos e sua aplicação deverá invocar essa TDA e obter os resultados das operações.

(c) Marçal, J.

• EXEMPLO PRÁTICO 02 – CADASTRO ALUNOS

- **CENÁRIO –** Desenvolva uma aplicação (menu) capaz de realizar o cadastro de alunos, defina ao menos 3 propriedades importantes para o aluno, além do RA que será o identificador único.
  - ETAPA 1 – Criar a struct Aluno, com os atributos.
  - ETAPA 2 – Crie um vetor do tipo do struct definido;
  - ETAPA 3 – Criar o menu com as opções Cadastrar, Imprimir Lista e Sair.
  - ETAPA 4 – Codificar cada um das opções do menu.